

TEMA 2 – Funciones en JavaScript

Objetivo del tema

En este tema aprenderemos el correcto uso de las funciones en JavaScript

2.1 Operadores Básicos

JavaScript dispone de una serie de operadores básicos para realizar operaciones sobre cadenas de texto, hacer comparaciones, etc. Las listamos a continuación:

Ejemplos de Operadores con **Cadenas de Texto**.

1. Abre el fichero **10. OperadoresCadena.html**
2. Lee el fichero.
3. Abre **10. OperadoresCadena.html** con el navegador.
4. Observa lo que sucede.

Fíjate en que son similares a los de Java.

Ejemplos de Operadores con **Cadenas de Texto**.

1. Abre el fichero **11. OperadoresComparacion.html**
2. Lee el fichero.
3. Abre **11. OperadoresComparacion.html** con el navegador.
4. Observa lo que sucede.

Tienes que quedarte con el operador de **igualdad/desigualdad estricta**. JavaScript considera iguales dos variables, aunque sean de tipos diferentes. El operador estricto los considera diferentes a demás por tipo.

Ejemplos de Operadores con **Cadenas Lógicos**.

1. Abre el fichero **12. OperadoresLogicos.html**
2. Lee el fichero.
3. Abre **12. OperadoresLogicos.html** con el navegador.
4. Observa lo que sucede.

Similares a los de Java.

2.2 Operador ternario

El operador ternario es una forma simplificada de realizar comparaciones tipo if-then-else más una asignación. Es muy cómoda de escribir y facilita la lectura, ya que evita andar repitiendo {...} y tal.

Ejemplos de operador ternario.

5. Abre el fichero **13. OperadorTernario.html**
6. Lee el fichero.
7. Abre **13. OperadorTernario.html** con el navegador.
8. Observa lo que sucede.

Si te fijas, la estructura es una simple línea.

```
var puedeVotar = (edad > 18) ? "Puede votar" : "No puede votar";
```

Esta línea se puede escribir así:

```
if ((edad > 18) {  
    puedeVotar = "Puede votar";  
} else {  
    puedeVotar = "No puede votar";  
}
```

2.3 Operador tipos

Dado que JavaScript tiene un tipado de datos débil, se hace necesario preguntar sobre el tipo de dato.

Ejemplos de operador ternario.

1. Abre el fichero **14. OperadoresTipos.html**
2. Lee el fichero.
3. Abre **14. OperadoresTipos.html** con el navegador.
4. Observa lo que sucede.

Observa cómo la instrucción **instanceof** permite consultar el tipo de dato de una variable.

Adicionalmente, tienes un ejemplo con los tipos de datos de JavaScript en

1. Abre el fichero **15. TiposDatos.html**
2. Lee el fichero.
3. Abre **15. TiposDatos.html** con el navegador.
4. Observa lo que sucede.

Como detalle, observa el tipo **Object**. Esta es una forma de JavaScript para definir y manejar **objetos anónimos**.

2.4 Funciones

Antes de empezar con las funciones en JavaScript. En los primeros ejemplos que hemos estado viendo, no usábamos funciones. Simplemente teníamos un código que se ejecutaba de principio a fin, a veces incluso con el Navegador sin terminar de 'pintar' la página.

Lo que sucede en un Navegador cuando 'pinta' una página es lo siguiente:

1. Coge las tres piezas: el HTML, el JS y el CSS
2. Copia el código del JS en su espacio correspondiente en el HTML
3. 'Pinta' la página línea a línea, y si encuentra código JavaScript, lo ejecuta
4. Cuando termina, le añade el CSS por encima.

Obviamente, estoy **simplificando** mucho las cosas. Esto es sólo un ejemplo ilustrativo.

Como sucede con todos los lenguajes de programación de alto nivel, JavaScript divide el código en **funciones**. Las ventajas de hacer esto son varias. Las más evidentes son:

1. Ordenar el código, lo que simplifica el **mantenimiento** del mismo.
2. Favorecer la **reutilización** del código. Lo escribes una vez, lo usas varias veces.

Por tanto, o habitual es tener uno o más ficheros de código JavaScript con una colección de funciones útiles que son utilizados por uno o más HTML.

El sentido común dicta que, excepto para fragmentos de código de pocas líneas, deberíamos definir siempre una función aparte. Pero como siempre, hay excepciones.

La forma de definir un método en JavaScript es la siguiente:

```
function nombre_funcion (<parámetros_separados_por_comas> { ... }
```

Vamos con unos ejemplos:

1. Abre el fichero **16. FuncionesIntroduccion.html**
2. Lee el fichero.
3. Abre **16. FuncionesIntroduccion.html** con el navegador.
4. Observa lo que sucede.

Cosas con las que nos tenemos que quedar:

- Las funciones no necesitan especificar el tipo de dato de **retorno**. Aquí entra de nuevo todo lo que ya hemos mencionado anteriormente a lo largo del tema.
- Es posible invocar una función simplemente sin indicar sus paréntesis. En ese caso, se retorna el código de la propia función.

2.5 Funciones Anónimas

Una **función anónima** no es algo exclusivo de JavaScript. Decimos que una función es anónima si se define **sin asignarle un nombre** identificador. Dado que no tienen nombre:

1. NO puedes invocarlas, porque no tienen nombre.
2. SI puedes almacenarlas en una variable
3. SI puedes usarla para inicializar objetos (*Constructor Function*)
4. SI puedes pasarlas como argumento a otra función (*Callback*)
5. SI puedes ejecutarlas inmediatamente. (*Immediately Invoked Function Expression*)
6. SI puedes definir las como funciones flecha (*Arrow Functions*)

Cuando decimos que 'no tiene nombre' estamos diciendo literalmente eso. Mira el ejemplo:

```
// Función tradicional (CON nombre)
function sumar(a, b) {
  return a + b;
}

// Función anónima (SIN nombre)
function(a, b) {
  return a + b;
}
```

Vamos con unos ejemplos:

1. Abre el fichero **17. FuncionesAnonimas.html**
2. Lee el fichero.
3. Abre **17. FuncionesAnonimas.html** con el navegador.
4. Observa lo que sucede.

Lo que tienes aquí son ejemplos sencillos de funciones anónimas. Quédate con esto:

1. Fíjate en el primer bloque que, cuando definimos una función anónima, escribimos el paso de parámetros y el cuerpo de la función {...}. Luego, la podemos asignar a una variable.
2. En el segundo bloque estamos usando la función anónima de miFuncion como un *Constructor* de una clase. Lo que nos devuelve es un objeto de tipo (**typeof**) function.
3. El tercer bloque es una *Immediately Invoked Function Expression*. Es decir, una función JavaScript que se ejecuta inmediatamente en cuanto son definidas.

Y lo que no viene en el ejemplo:

4. *Callback*: pasar el retorno de una función como parámetro de otra.

```
const numeros = [1, 2, 3];

// Se le pasa una función anónima a .forEach()
numeros.forEach(function(numero) {
  console.log(numero * 2);
});
```

5. *Arrow*: Introducidas en JavaScript 6, que ofrecen una sintaxis más corta (no es obligatorio usarlas).

```
const restar = (a, b) => a - b;
```

Genial. Pero... ¿cuándo se usa una función anónima en lugar de una convencional?

1. Cuando tengas que operar sobre **cada elemento de un Array**. En este caso, es mucho más limpio hacer algo como esto:

```
const numeros = [1, 2, 3, 4];

const dobles = numeros.map(function(n) {
  return n * 2;
});
```

2. **Manejo de Eventos**. Es raro que tengas más de un componente de la página que haga una cosa, por tanto, podríamos usar funciones anónimas:

```
boton.addEventListener("click", function() {
  console.log("¡Hiciste clic!");
});
```

3. **Asignación a Variables**. Normalmente cuando su inicialización es compleja:

```
const saludar = function(nombre) {
  return `Hola, ${nombre}`;
};

console.log(saludar("Ana"));
```

Importante – ¿Y esto, de memoria?

Claro que no. Ahora no tienes experiencia con JavaScript. Con la práctica irás desarrollando las destrezas necesarias para saber qué es más conveniente. Dicho esto, el uso de funciones anónimas es muy habitual. Deberías intentar utilizarlas cuando sea conveniente.

2.6 Parámetros de una función

Una función en JavaScript puede tener cero o más parámetros (obvio). Pero **no caigas en el error** de creer que es igual que con Java. Dado que es muy flexible, hay varias cosas a tener en cuenta.

- No se especifica el tipo de los parámetros. Como las variables.
- No se **verifican** los tipos de los argumentos. ¿Recuerdas el problema de los tipos de datos?
- No se **comprueba** el número de los argumentos recibidos. Fuente de innumerables problemas.

Vamos con unos ejemplos:

1. Abre el fichero **18. FuncionesParametros.html**
2. Lee el fichero.
3. Abre **18. FuncionesParametros.html** con el navegador.
4. Observa lo que sucede.

Tenemos que quedarnos con lo siguiente:

1. Si pasamos **parámetros de menos**, toda variable que no pasemos, será **undefined**. Y no Null.
2. Si pasamos **parámetros de más**, aún podemos recuperarlos. Todos los valores que nos llegan a una función pueden capturarse a través de un objeto (incluido en la función) llamado **arguments**.

Importante – Sobrecarga de funciones

NO. NUNCA. En JavaScript no existe el concepto de sobrecarga de funciones. Si tú defines una función sumar (a) y debajo añades sumar (a, b), la última definición sobrescribirá por completo a la primera, sin importar si tienen un número diferente de parámetros.

2.7 Parámetros de una variable

Ya lo hemos visto anteriormente. Puedes hacer un repaso en:

1. Abre el fichero **19. Ámbito variables.html**
2. Lee el fichero.
3. Abre **19. Ámbito variables.html** con el navegador.
4. Observa lo que sucede.

2.8 Break y Continue

Equivalentes a java. Puedes hacer un repaso en:

1. Abre el fichero **20. Etiquetas y saltos.html**
2. Lee el fichero.
3. Abre **20. Etiquetas y saltos.html** con el navegador.
4. Observa lo que sucede.

Ejercicio 9 – Entre números

Realiza una página web que llame a una función llamada **numerosentre** definida en un js a parte. La web debe solicitar dos números. Se deben mostrar todos los números entre el primero y el segundo (sin incluirlos), o un aviso de error si el primer número es mayor que el segundo. Se debe ofrecer la oportunidad de volver a solicitar el par de números al usuario. Muestra e resultado en la propia página web (HTML).

Ejercicio 10 – Ordenar números

Realiza una página web que llame a una función llamada **ordenar** definido en un js a parte. La web debe solicitar tres números. Se deben mostrar ordenados en la propia página web (HTML).

Ejercicio 11 – Perímetro Cuadrado

Realiza una página web que llame a una función llamada **perímetro** definido en un js a parte. La web debe solicitar un número, que será el lado de un cuadrado. Se deben mostrar el perímetro del cuadrado. Se debe ofrecer la oportunidad de volver a solicitar el lado del cuadrado al usuario. Muestra e resultado en la propia página web (HTML).

Ejercicio 12 – Perímetro

Realiza una página web que permita seleccionar entre cuadrado, círculo y triángulo. A continuación, solicita los parámetros necesarios para calcular las áreas de la figura seleccionada, cada una de ellas obtenida mediante su correspondiente función JavaScript. Controla que la opción seleccionada es válida. Muestra el resultado en un alert (). El programa termina si seleccionamos 'salir'.

Ejercicio 13 – Calculadora

Realiza una página web que funcione como una calculadora con las cuatro operaciones básicas. Controla que la opción seleccionada es válida. Muestra el resultado en un alert (). El programa termina si seleccionamos 'salir'.

Ejercicio 14 – DNI

Realiza una página web que verifique DNI. Solicita el DNI al usuario y comprueba que está formado por 8 números y una letra mayúscula al final. 'salir'. Comprueba que la letra corresponde con la cifra numérica. Puedes mirar en Internet cómo se hace.

Ejercicio 15 – Textos

Realiza una página web que verifique cadenas de texto. Crea una función que determine si esa cadena está formada sólo por mayúsculas, sólo por minúsculas o por una mezcla de ambas. El usuario debe poder volver a realizar la comprobación.

Ejercicio 16 – Palíndromo

Realiza una página web que verifique si una cadena de texto es un palíndromo, es decir, si se lee de la misma forma desde la izquierda y desde la derecha.

Ejercicio 17 – Facturas

Elabora una página y los scripts necesarios para el siguiente escenario.

La página se corresponde con una Factura. La factura tendrá varias partes diferenciadas:

- 1- Cliente: Le pediremos al usuario el nombre del cliente, el DNI, y su número de teléfono
- 2- Empresa: Rellenaremos nuestros datos, que son el nombre de la empresa, NIE y la fecha de factura.
- 3- Importes: Añadiremos de 1 a N productos con su importe.
- 4- Cuando hayamos terminado, mostraremos en el HTML el resultado:
 - a. Los datos del Cliente
 - b. La empresa
 - c. Una tabla con todos los productos e importes
 - d. El importe total.
 - e. El importe tras meterle un 16% de IVA

La página tiene que ser a **prueba de errores**. Esto significa verificar todo lo que nos introduce el usuario, lo que incluye tipos de dato, unidentified, etc. Por regla, volvemos a pedir el dato inmediatamente.

Maqueta un poco la página con CSS, anda...